Figures 3.3 and 3.4 illustrate the difference between a nonstreaming and a streaming response flow. In the nonstreaming version in figure 3.3, the client sends a request to the server, and the server processes the request, generates the complete response, and sends it back to the client in one go. The client then receives the entire response, processes it, and updates the user interface accordingly.

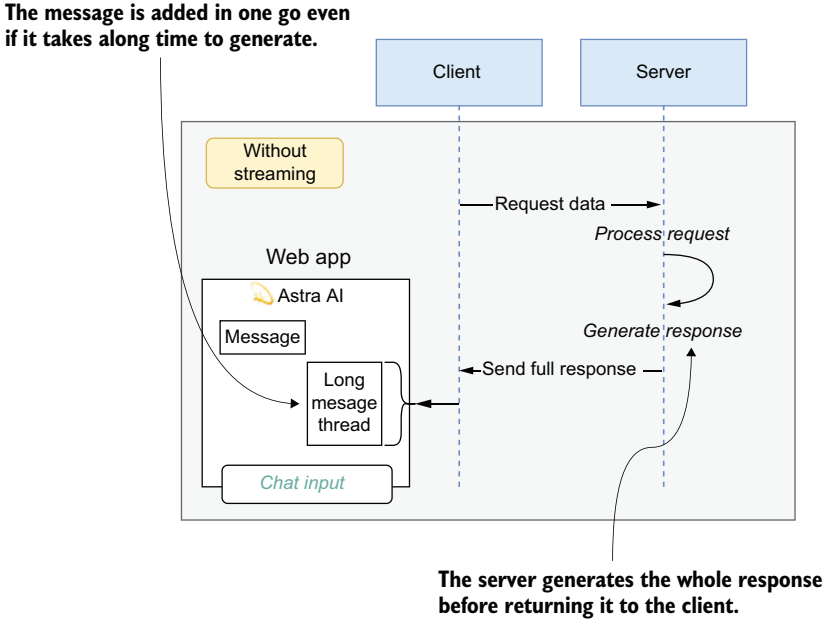


Figure 3.3 Without streaming, the client sends a request and waits for the server to generate and send the full response before displaying it.

Now let's compare the streaming version in figure 3.4. Here, the client sends a request to the server, and the server processes the request and starts generating the response. Now, instead of waiting for the entire response to be ready, the server sends the response in smaller chunks. Each chunk is sent to the client as soon as it is generated. The client receives the chunks one by one, processes them, and updates the user interface incrementally. This process continues until the server sends the final chunk, indicating the completion of the response.

If you think all of that sounds very complicated and difficult to implement, you are right. To handle the challenges associated with streaming, developers need to implement appropriate mechanisms for asynchronous processing, connection management, data buffering, and error handling. Here is where the Vercel AI SDK comes to the rescue, as it simplifies this process by abstracting away many of the low-level details and providing a high-level API for handling streaming responses.